

Kiste Phase 5 Spec

Python Library + CLI

Turn Kiste from a CLI prototype into an installable Python package with a stable public API and a CLI wrapper.

One-sentence spec: Kiste Phase 5 turns Kiste into an installable Python package with a stable public library API and a CLI wrapper, so scanner, token, workspace, GitOps, and secret-management capabilities can be used from both terminal commands and Python code.

Phase	Focus
Phase 1	Local Docker repo scanner + token inspector
Phase 2	GitHub/Hugging Face remote repo scanner + Kiste CI testing
Phase 3	Multi-repo workspace + system graph + token access matrix
Phase 4	GitOps repo generation + SOPS encrypted secrets + Vault-ready backend
Phase 4.5	Review, refactor, and architecture checkpoint
Phase 5	Python library + CLI wrapper

1. Phase 5 Goal

Phase 5 makes Kiste reusable. The product should work as a command-line tool and as a Python library. The CLI remains important for developers, but the core logic must live in importable Python modules.

```
# CLI usage
kiste scan .
kiste check --repo .
kiste workspace scan
kiste gitops generate --env dev

# Python library usage
from kiste.scanner import scan_repo
from kiste.workspace import load_workspace
from kiste.gitops import generate_gitops_repo

repo_spec = scan_repo(".")
workspace = load_workspace("kiste.workspace.yaml")
generate_gitops_repo(workspace, output="./robot-platform-gitops")
```

The design rule is simple: the CLI should not contain the main logic. The CLI calls the Python library.

2. Why Phase 5 Matters

- Other tools can import Kiste instead of shelling out to commands.
- A future FastAPI server, MCP server, VS Code extension backend, Jenkins plugin, or GitHub Action can reuse the same Python API.
- Tests can target the real product logic instead of only testing CLI output.
- The codebase becomes easier to maintain because commands are thin wrappers around stable APIs.
- Provider-specific SDKs can become optional dependencies instead of always being installed.

3. Architecture Principle

The architecture should be library-first. Kiste should expose stable modules for scanning, providers, token checks, workspaces, GitOps generation, and secret backends. The CLI imports these modules and only handles user input/output.

```
Kiste Python Library
-> core domain logic
-> scanner / providers / tokens / workspace / gitops / secrets
-> CLI wrapper
-> user commands
```

Bad design: CLI command contains scanning, token, and GitOps generation logic directly. Good design: CLI command validates arguments, calls a library function, then formats the result.

4. Recommended Package Structure

```
kiste/
  __init__.py

  core/
    models.py
    errors.py
    config.py

  scanner/
    local.py
    remote.py
    spec_builder.py

  providers/
    base.py
    github.py
    huggingface.py

  tokens/
```

```
manager.py
checker.py

workspace/
  manager.py
  graph.py
  connection_inference.py

gitops/
  generator.py
  validator.py

secrets/
  base.py
  sops.py
  scanner.py

cli/
  main.py
  scan.py
  token.py
  workspace.py
  gitops.py
  secrets.py
```

The most important dependency direction is: CLI imports library. The library must not import the CLI.

5. Public Python API

Phase 5 should define a small, stable public API. Internal modules can change, but these functions should remain predictable for users.

5.1 Repo scanning

```
from kiste.scanner import scan_repo

spec = scan_repo(".")
print(spec.app.framework)
print(spec.app.port)
```

5.2 Token check

```
from kiste.tokens import check_token

result = check_token(
    token_name="github-dev",
    repo="owner/repo",
    target="scan"
)
```

5.3 Workspace

```
from kiste.workspace import load_workspace, scan_workspace

workspace = load_workspace("kiste.workspace.yaml")
report = scan_workspace(workspace)
```

5.4 GitOps

```
from kiste.gitops import generate_gitops_repo, validate_gitops_repo

generate_gitops_repo(
    workspace=workspace,
    output="./robot-platform-gitops",
    env="dev"
)

validation = validate_gitops_repo("./robot-platform-gitops")
```

5.5 Secrets

```
from kiste.secrets import SopsSecretBackend

backend = SopsSecretBackend(config_file=".sops.yaml")
backend.encrypt_secret(
    name="api-secret",
    data={
        "DATABASE_URL": "...",
        "SECRET_KEY": "..."
    }
)
```

6. CLI Wrapper Design

Every CLI command should map cleanly to a library function. The CLI layer should handle parsing, display, exit codes, and friendly error messages. The library layer should return typed results.

```
# CLI layer example
@app.command()
def scan(repo: str = "."):
    spec = scan_repo(repo)
    print_scan_report(spec)

# The scan_repo function belongs to the library, not the CLI.
```

Stable command tree

```
kiste
  scan
  check
```

- repo
 - summary
 - services
 - env
 - ports
 - graph

- token
 - add
 - list
 - inspect
 - check

- workspace
 - init
 - add
 - scan
 - graph
 - check
 - token

- gitops
 - init
 - generate
 - validate

- secrets
 - init
 - add
 - scan

7. Library-First Data Models

Phase 5 should replace loose dictionaries with typed data models. Pydantic is a good fit because it gives validation, serialization, and readable errors.

```
from pydantic import BaseModel
from typing import Literal

class AppSpec(BaseModel):
    language: str | None = None
    framework: str | None = None
    port: int | None = None
    start_command: str | None = None

class ServiceSpec(BaseModel):
    name: str
    type: Literal["web", "database", "cache", "worker", "queue", "model"]
    public: bool = False
    port: int | None = None

class RepoSpec(BaseModel):
    name: str
    provider: str = "local"
    repo: str | None = None
    app: AppSpec
    services: list[ServiceSpec] = []
```

Core models to stabilize

Model	Purpose
RepoSpec	One scanned repo: app, Docker, services, env vars, readiness
WorkspaceSpec	Many repos connected into one product/system
Connection	Relationship such as frontend -> api or api -> postgres
TokenMetadata	Token provider, scopes, expiry, and secret reference
TokenAccessResult	What a token can do for a repo/action
GitOpsSpec	Generated GitOps repo structure and environment overlays
SecretBackendConfig	SOPS now; Vault and other backends later
ReadinessReport	Structured pass/warn/fail output for users and CI

8. pyproject.toml

The package should be installable through pip and usable as a console script. Core dependencies should stay light. Provider SDKs should be optional.

```
[project]
name = "kiste"
version = "0.5.0"
description = "Repo scanner, token inspector, workspace graph, and GitOps generator"
requires-python = "3.10+"
dependencies = [
    "typer==0.12.*",
    "pydantic==2.*",
    "PyYAML==6.*",
    "requests==2.31.*"
]

[project.scripts]
kiste = "kiste.cli.main:app"

[project.optional-dependencies]
dev = [
    "pytest==8.*",
    "ruff==0.6.*",
    "mypy==1.10.*"
]
```

```
github = ["PyGithub==2.*"]  
hf = ["huggingface_hub==0.24.*"]  
sops = ["subprocess-tee==0.4.*"]
```

Recommended installs: pip install kiste, pip install "kiste[github]", pip install "kiste[hf]", and pip install "kiste[dev]".

9. Optional Dependency Strategy

Kiste should avoid loading every SDK at once. This keeps startup fast and avoids forcing users to install cloud/provider libraries they do not need.

- Core install: local scanning, core models, CLI, YAML/JSON support.
- github extra: GitHub provider integration.
- hf extra: Hugging Face provider integration.
- sops extra: SOPS command integration and secret backend support.
- dev extra: tests, linting, type checking.

If a user runs a command that needs a missing extra, Kiste should print a helpful message such as: `pip install "kiste[hf]"`.

10. Testing Plan

Because Kiste becomes a library and a CLI, tests should cover both API behavior and CLI behavior.

```
tests/
  fixtures/
    fastapi_docker_repo/
    nextjs_docker_repo/
    hf_space_repo/

  test_scanner_api.py
  test_workspace_api.py
  test_gitops_api.py
  test_secret_backend_api.py
  test_cli_scan.py
  test_cli_workspace.py
```

Example API test

```
def test_scan_repo_returns_repo_spec():
    spec = scan_repo("tests/fixtures/fastapi_docker_repo")
    assert spec.app.framework == "fastapi"
    assert spec.app.port == 8000
```

Example CLI test

```
def test_cli_scan(runner):
    result = runner.invoke(app, ["scan", "tests/fixtures/fastapi_docker_repo"])
    assert result.exit_code == 0
    assert "FastAPI" in result.output
```

11. What Phase 5 Should Not Do Yet

Phase 5 is about packaging, API stability, and clean architecture. It should not add too many new product features.

- No real deployment engine yet.
- No full Vault integration yet.
- No full cloud provisioning yet.
- No registry expansion yet.
- No Kubernetes apply automation yet.
- No complex runtime controller yet.

12. Acceptance Criteria

Phase 5 is complete when Kiste can be installed, imported, tested, and used from the terminal.

```
pip install -e .
```

```
kiste --help
kiste scan .
pytest
```

Python users should be able to import the main APIs:

```
from kiste.scanner import scan_repo
from kiste.workspace import load_workspace
from kiste.gitops import generate_gitops_repo
```

Required deliverables:

- Installable Python package.
- Working CLI entrypoint.
- Stable public Python API.
- Typed data models.
- CLI built on top of library functions.
- Tests for both API and CLI.
- Optional dependencies for GitHub, Hugging Face, SOPS, and dev tools.

13. Suggested Phase 5 Milestones

Step	Deliverable
5.1	Move CLI logic into library modules
5.2	Add Pydantic models for RepoSpec, WorkspaceSpec, TokenMetadata, and reports
5.3	Expose public API imports through kiste.scanner, kiste.workspace, kiste.gitops, and kiste.secrets
5.4	Update CLI commands to call library APIs only
5.5	Add pyproject optional dependencies
5.6	Add API and CLI tests
5.7	Prepare package for pip/uv publishing

14. Final Principle

Kiste should become a composable developer platform. The CLI is one interface. Python is the product core. This makes it easier to add future interfaces such as an API server, MCP server, VS Code extension backend, CI plugin, or platform UI without rewriting the scanner and workspace logic.